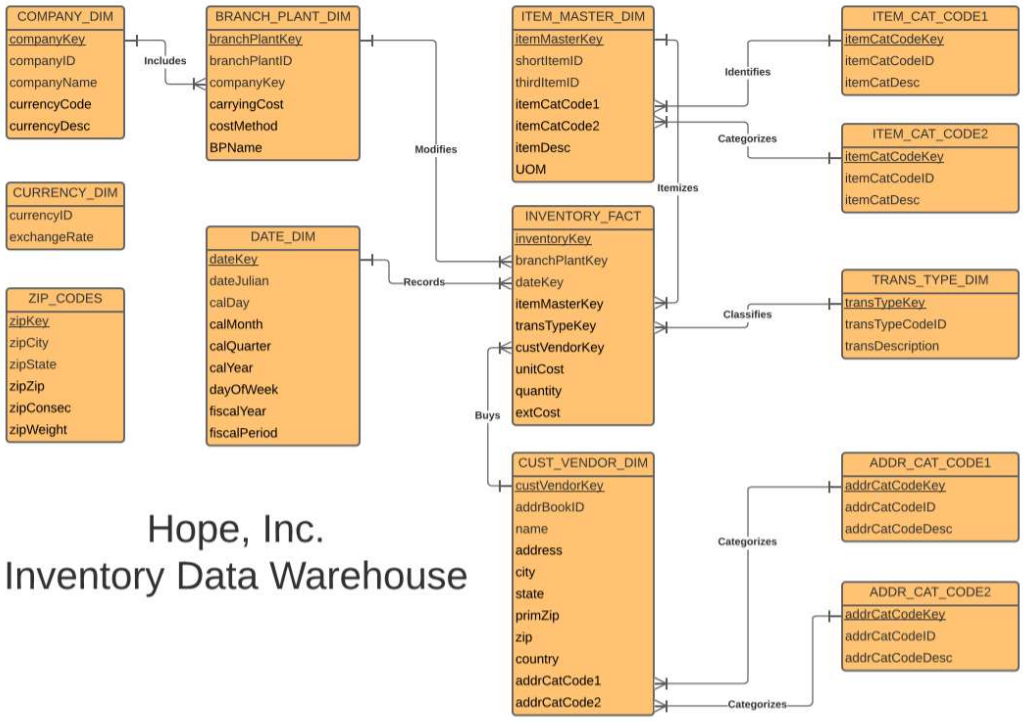
Analytic Function Assignment



REFER TO THE ABOVE ERD.
You may re-generate the data warehouse using these steps.

IMPORTANT NOTE: DO NOT USE INNER JOIN, FULL JOIN, LEFT JOIN, RIGHT JOIN IN YOUR SOLUTIONS. Stick with the WHERE clause in connecting the primary and foreign keys of the tables.

Query 1: Ranking within the entire result

Use the RANK function to rank customers in descending order by the sum of extended cost for shipments (transaction type 5). You should use the entire result as a single partition. The result should include the customer name, sum of the extended cost, and rank.

```
SELECT NAME,
        SUM(EXTCOST) AS TOTAL_COST,
         RANK () OVER (ORDER BY SUM(EXTCOST) DESC) AS CUST_RANK
FROM INVENTORY_FACT F JOIN CUST_VENDOR_DIM C ON F.CUSTVENDORKEY
= C.CUSTVENDORKEY
WHERE TRANSTYPEKEY = 5
GROUP BY NAME;
```

Filter table Total:20

NAME	TOTAL_COST	CUST_RANK
Customer 8	44576414.00	1
Customer 18	44471146.00	2
Customer 12	42977058.00	3
Customer 19	42641911.00	4
Customer 10	42520307.00	5
Customer 1	42393994.00	6

Query 2: Ranking within a partition

Use the RANK function to rank customers in descending order by the sum of extended cost for shipments (transaction type 5). You should partition the rank values by customer state. The result should include the customer state, customer name, sum of the extended cost, and rank. You should order the result by customer state.

```
SELECT STATE,
                NAME,
                SUM(EXTCOST) AS TOTAL_COST,
                RANK () OVER (PARTITION BY STATE ORDER BY SUM(EXTCOST)
DESC) AS CUST_STATE_RANK
FROM INVENTORY_FACT F JOIN CUST_VENDOR_DIM C ON F.CUSTVENDORKEY
= C.CUSTVENDORKEY
WHERE TRANSTYPEKEY = 5
GROUP BY STATE, NAME
ORDER BY STATE;
```

Filter table Total:

STATE	NAME	TOTAL_COST	CUST_STATE_RANK
CA	Customer 20	40287693.00	1
CA	Customer 14	38687623.00	2
CA	Customer 5	38657564.00	3
CO	Customer 18	44471146.00	1
CO	Customer 3	40680456.00	2
DC	Customer 12	42977058.00	1

Query 3: Ranking and dense ranking within the entire result

Use both RANK and DENSE_RANK functions to rank customers in descending order by the count of inventory transactions for shipments (transaction type 5). You should use the entire result as a single partition. The result should include the customer name, count of transactions, rank, and dense rank.

```
SELECT NAME,
                COUNT(*) AS TOTAL_TRANSACTIONS,
                RANK () OVER (ORDER BY COUNT(*) DESC) AS CUST_STATE_RANK,
                DENSE_RANK () OVER (ORDER BY COUNT(*) DESC) AS
CUST_STATE_DENSE_RANK
FROM INVENTORY_FACT F JOIN CUST_VENDOR_DIM C ON F.CUSTVENDORKEY
= C.CUSTVENDORKEY
WHERE TRANSTYPEKEY = 5
GROUP BY NAME;
```

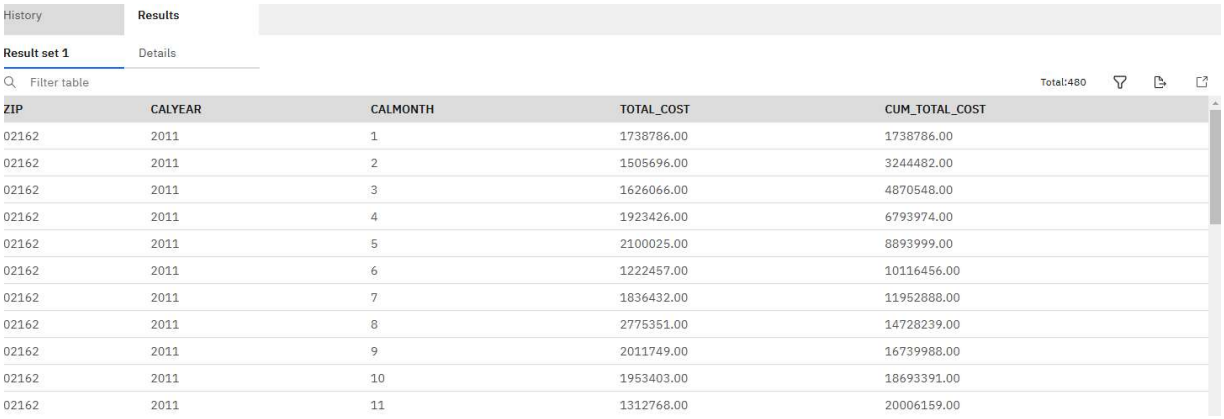
Filter table Total:20

NAME	TOTAL_TRANSACTIONS	CUST_STATE_RANK	CUST_STATE_DENSE_RANK
Customer 18	457	1	1
Customer 8	446	2	2
Customer 12	438	3	3
Customer 19	428	4	4
Customer 13	426	5	5
Customer 9	424	6	6
Customer 1	423	7	7
Customer 17	419	8	8
Customer 20	419	8	8
Customer 10	414	10	9

Query 4: Cumulative extended costs for the entire result

Calculate the cumulative sum of extended cost ordered by customer zip code, calendar year, and calendar month for shipments (transaction type 5). The result should include the customer zip code, calendar year, calendar month, sum of the extended cost, and cumulative sum of the extended cost. Note that the cumulative extended cost is the sum of the extended cost in the current row plus the cumulative sum of extended costs in all previous rows.

```
SELECT ZIP, CALYEAR,CALMONTH, SUM(EXTCOST) AS TOTAL_COST,
SUM(SUM(EXTCOST)) OVER
  (ORDER BY ZIP, CALYEAR, CALMONTH ROWS UNBOUNDED PRECEDING ) AS
CUM_TOTAL_COST
FROM INVENTORY_FACT F JOIN CUST_VENDOR_DIM C ON F.CUSTVENDORKEY
= C.CUSTVENDORKEY
                JOIN DATE_DIM D ON D.DATEKEY = F.DATEKEY
WHERE TRANSTYPEKEY = 5
GROUP BY ZIP, CALYEAR, CALMONTH
ORDER BY ZIP, CALYEAR, CALMONTH;
```



History Results

Result set 1 Details

Filter table Total:480

ZIP	CALYEAR	CALMONTH	TOTAL_COST	CUM_TOTAL_COST
02162	2011	1	1738786.00	1738786.00
02162	2011	2	1505696.00	3244482.00
02162	2011	3	1626066.00	4870548.00
02162	2011	4	1923426.00	6793974.00
02162	2011	5	2100025.00	8893999.00
02162	2011	6	1222457.00	10116456.00
02162	2011	7	1836432.00	11952888.00
02162	2011	8	2775351.00	14728239.00
02162	2011	9	2011749.00	16739988.00
02162	2011	10	1953403.00	18693391.00
02162	2011	11	1312768.00	20006159.00

Query 5: Cumulative extended costs for a partition

Calculate the cumulative sum of extended cost ordered by customer zip code, calendar year, and calendar month for shipments (transaction type 5). Restart the cumulative extended cost after each combination of zip code and calendar year. The result should include the customer zip code, calendar year, calendar month, sum of the extended cost, and cumulative sum of the extended cost. Note that the cumulative extended cost is the sum of the extended cost in the current row plus the cumulative sum of extended costs in all previous rows of the store zip code and years. The value of cumulative extended cost resets in each partition (new value for zip code and year).

```
SELECT ZIP, CALYEAR,CALMONTH, SUM(EXTCOST) AS TOTAL_COST,
SUM(SUM(EXTCOST)) OVER
  (PARTITION BY ZIP, CALYEAR ORDER BY ZIP, CALYEAR, CALMONTH ROWS
UNBOUNDED PRECEDING ) AS CUM_TOTAL_COST
FROM INVENTORY_FACT F JOIN CUST_VENDOR_DIM C ON F.CUSTVENDORKEY
= C.CUSTVENDORKEY
                JOIN DATE_DIM D ON D.DATEKEY = F.DATEKEY
WHERE TRANSTYPEKEY = 5
GROUP BY ZIP, CALYEAR, CALMONTH
ORDER BY ZIP, CALYEAR, CALMONTH;
```



Filter table Total:480

ZIP	CALYEAR	CALMONTH	TOTAL_COST	CUM_TOTAL_COST
02162	2011	1	1738786.00	1738786.00
02162	2011	2	1505696.00	3244482.00
02162	2011	3	1626066.00	4870548.00
02162	2011	4	1923426.00	6793974.00
02162	2011	5	2100025.00	8893999.00
02162	2011	6	1222457.00	10116456.00
02162	2011	7	1836432.00	11952888.00
02162	2011	8	2775351.00	14728239.00
02162	2011	9	2011749.00	16739988.00

Query 6: Ratio to report applied to the entire result

Calculate the ratio to report the sum of extended cost for adjustments (transaction type 1). You should sort on descending order by sum of extended cost. The result should contain the second item id, sum of extended cost, and ratio to report.

```
SELECT SECONDITEMID,
        SUM(EXTCOST) AS TOTAL_COST,
        RATIO_TO_REPORT (SUM(EXTCOST)) OVER () AS RATIO_TO_REPORT
FROM INVENTORY_FACT F JOIN ITEM_MASTER_DIM I ON F.ITEMMASTERKEY =
I.ITEMMASTERKEY
WHERE TRANSTYPEKEY = 1
GROUP BY SECONDITEMID
ORDER BY 2 DESC;
```

Filter table

SECONDITEMID	TOTAL_COST	RATIO_TO_REPORT
Second Part 20	2843861.00	0.066841666891715978320147794054895 25
Second Part 19	2707968.00	0.063647658943044802252871705604193 24
Second Part 11	2699451.00	0.063447476698934859823423607134554 41
Second Part 1	2529470.00	0.059452269696932731832344914406166 79
Second Part 9	2499707.00	0.058752724771319931958250308940408 85
Second Part 10	2498878.00	0.058733240084180429519127087896457 86
Second Part 3	2444422.00	0.057453314724869518994526213144473 36
Second Part 12	2264090.00	0.053214819427836040282863128346198 28
Second Part 4	2196283.00	0.051621094239816448136146301654781 13
Second Part 18	2174922.00	0.051119028616189293006941084863662 78

Query 7: Ratio to report applied to a partition

Calculate the ratio to report the sum of extended cost for adjustments (transaction type 1) with partitioning on the calendar year. You should sort on ascending order by calendar year and descending order by sum of extended cost. The result should contain the calendar year, second item id, sum of extended cost, and ratio to report.

```
SELECT CALYEAR,
            SECONDITEMID,
            SUM(EXTCOST) AS TOTAL_COST, RATIO_TO_REPORT
(SUM(EXTCOST)) OVER (PARTITION BY CALYEAR) AS RATIO_TO_REPORT
FROM INVENTORY_FACT F JOIN ITEM_MASTER_DIM I ON F.ITEMMASTERKEY =
I.ITEMMASTERKEY
            JOIN DATE_DIM D ON F.DATEKEY = D.DATEKEY
WHERE TRANSTYPEKEY = 1
GROUP BY CALYEAR, SECONDITEMID
ORDER BY 1,3 DESC;
```

CALYEAR	SECONDITEMID	TOTAL_COST	RATIO_TO_REPORT
2011	Second Part 20	1809863.00	0.083885322771819795310807754909134 43
2011	Second Part 10	1547769.00	0.071737530487786508022106429054550 58
2011	Second Part 4	1432612.00	0.066400119802870328085499797153643 61
2011	Second Part 18	1428655.00	0.066216716847946065348461211202013 33
2011	Second Part 3	1412170.00	0.065452653741570914673687117339838 63
2011	Second Part 11	1390063.00	0.064428016611292755379344933960336 30
2011	Second Part 9	1294296.00	0.059989312849798727191835643884577 48
2011	Second Part 15	1161102.00	0.053815905425441322340403392875872 82
2011	Second Part 12	1139753.00	0.052826400829869402913302869377931 20

Query 8: Cumulative distribution functions for carrying cost of all branch plants

Calculate the rank, percent_rank, and cume_dist functions of the carrying cost in the branch_plant_dim table. The result should contain the BPName, CompanyKey, CarryingCost, rank, percent_rank, and cume_dist.

```
SELECT BPNAME, COMPANYKEY, CARRYINGCOST,
                        RANK() OVER (ORDER BY CARRYINGCOST) AS RANK,
                        PERCENT_RANK() OVER (ORDER BY CARRYINGCOST) AS
PERCENT_RANK,
                        CUME_DIST () OVER (ORDER BY CARRYINGCOST) AS
CUME_DIST

FROM BRANCH_PLANT_DIM;
```

Result set 1 Details

Filter table Total:20

BPNAME	COMPANYKEY	CARRYINGCOST	RANK	PERCENT_RANK	CUME_DIST
Branch Plant 3	1	0.00	1	0	0.1
Branch Plant 8	2	0.00	1	0	0.1
Branch Plant 6	2	0.01	3	0.105263157894736842105263157894736 8	0.15
Branch Plant 9	3	0.03	4	0.157894736842105263157894736842105 3	0.2
Branch Plant 13	4	0.05	5	0.210526315789473684210526315789473 7	0.25
Branch Plant 16	4	0.08	6	0.263157894736842105263157894736842 1	0.3
Branch Plant 15	4	0.10	7	0.315789473684210526315789473684210 5	0.35
Branch Plant 10	3	0.11	8	0.368421052631578947368421052631578 9	0.4
Branch Plant 2	1	0.14	9	0.421052631578947368421052631578947 4	0.55
Branch Plant 18	5	0.14	9	0.421052631578947368421052631578947 4	0.55
Branch Plant 19	5	0.14	9	0.421052631578947368421052631578947 4	0.55

Query 9: Determine worst performing plants

Determine the branch plants with the highest carrying costs (top 15%). The result should contain BPName, CompanyKey, CarryingCost, and cume_dist.

```
SELECT * FROM (
```

```
SELECT BPNAME,COMPANYKEY, CARRYINGCOST,CUME_DIST () OVER ( ORDER
BY CARRYINGCOST) AS CUME_DIST
FROM BRANCH_PLANT_DIM)
WHERE CUME_DIST >= 0.85;
```

Filter table Total:4

BPNAME	COMPANYKEY	CARRYINGCOST	CUME_DIST
Branch Plant 1	1	0.21	0.95
Branch Plant 7	2	0.21	0.95
Branch Plant 17	5	0.21	0.95
Branch Plant 20	5	0.23	1

Query 10: Cumulative distribution of extended cost for Colorado inventory

Calculate the cumulative distribution of extended cost for Colorado inventory (condition on customer state). The result should contain the extended cost and cume_dist, ordered by extended cost. You should eliminate duplicate rows in the result.

```
SELECT DISTINCT EXTCOST, CUME_DIST() OVER (ORDER BY EXTCOST) AS
CUME_DIST
FROM INVENTORY_FACT F JOIN CUST_VENDOR_DIM C ON C.CUSTVENDORKEY
= F.CUSTVENDORKEY
WHERE STATE = 'CO'
ORDER BY 1;
```

Total:1712

EXTCOST	CUME_DIST
28458.00	0.00058139534883720930232558139534883720
29535.00	0.00116279069767441860465116279069767441
30576.00	0.00174418604651162790697674418604651162
30957.00	0.00232558139534883720930232558139534883
32450.00	0.00290697674418604651162790697674418604
32966.00	0.00348837209302325581395348837209302325
34408.00	0.00406976744186046511627906976744186046
34485.00	0.00465116279069767441860465116279069767